

Assignment 5

- 关于排序算法的“稳定性 (Stability)”，下列说法中最为准确且严谨的是 ()
 - 某种排序算法被判定为“不稳定”，意味着对于任意输入序列，该算法都会改变相同关键字元素的相对前后顺序
 - 在基于比较的内部排序算法中，只要该算法的最坏时间复杂度达到了 $O(n \log_2 n)$ ，它就必然是一种不稳定的排序算法
 - 稳定排序算法在排序过程中，绝对不会发生跨越多个元素的“长距离”交换操作
 - 强制将一个不稳定的排序算法改为稳定排序，其空间复杂度必然要上升到 $O(n)$
- 对一个包含 n 个元素的无序序列进行折半插入排序。与传统的直接插入排序相比，折半插入排序在执行全过程中 ()
 - 极大地减少了元素的移动次数，但关键字的比较次数保持不变
 - 元素的移动次数没有减少，但关键字的比较次数大幅减少，且该比较次数与初始序列的排列状态基本无关
 - 无论是关键字的比较次数还是元素的移动次数，都得到了显著的减少
 - 当初始序列已经处于严格升序（正序）状态时，折半插入排序的关键字比较次数少于直接插入排序
- 已知一个初始关键字序列为 {50, 26, 38, 80, 70, 56, 12, 40}。现采用希尔排序 (Shell Sort) 对其进行升序排列，若第一趟排序选定的增量（步长） $d_1 = 4$ ，则第一趟希尔排序彻底结束后的关键字序列是 ()
 - 12, 26, 38, 40, 50, 56, 70, 80
 - 26, 38, 50, 80, 12, 40, 56, 70
 - 50, 26, 12, 40, 70, 56, 38, 80
 - 40, 26, 12, 50, 70, 56, 38, 80
- 在数据结构常用的内部排序算法中，往往有些算法具备特殊的工程性质。下列五种排序算法中：
① 直接插入排序 ② 简单选择排序 ③ 起泡排序 ④ 快速排序 ⑤ 堆排序
具有“在整个排序的任何一趟彻底结束后，都至少能保证有一个元素被放置到了其最终排序后的绝对正确位置上”这一特征的算法组合是 ()
 - 仅 ②③④
 - 仅 ②④⑤
 - 仅 ②③④⑤
 - ①②③④⑤ 全都是
- 快速排序的时间性能在极大程度上取决于划分 (Partition) 的对称性。设有一个包含 n 个不同元素的无序序列，若每次划分选取的枢轴 (Pivot) 元素都恰好是当前待排序列中的中位数，则快速排序的空间复杂度为 ()
 - $O(1)$
 - $O(\log_2 n)$
 - $O(n)$
 - $O(n \log_2 n)$
- 对一组初始关键字序列 {15, 25, 40, 30, 10, 50} 构建初始大根堆（采用顺序存储，一维数组下标从 1 开始）。在自底向上的初次建堆调整彻底完成后，该一维数组中排在第四个位置（即数组下标为 4）的元素是 ()

- A. 15
 - B. 25
 - C. 30
 - D. 40
7. 在对包含 n 个元素的序列进行标准的递归型二路归并排序 (Merge Sort) 时, 系统的算法总空间复杂度被定义为 $O(n)$ 。关于这一空间复杂度的微观来源, 下列说法中正确的是 ()
- A. 它完全来自于递归调用时由操作系统分配的栈空间
 - B. 它完全来自于为了将两个有序子表合并为一个有序表而强制开辟的辅助数组空间
 - C. 它是递归调用栈空间 $O(\log_2 n)$ 与合并辅助数组空间 $O(n)$ 的代数加和, 根据渐进复杂度取大者法则最终记为 $O(n)$
 - D. 如果将递归实现改为非递归 (迭代) 实现, 归并排序的空间复杂度可以成功降至 $O(1)$
8. 已知一组包含 1000 个随机无序的三位十进制正整数 (例如 342, 105 等), 现采用最低位优先 (LSD) 的基数排序 (Radix Sort) 对其进行升序排列。在排序的全过程中, 该算法一共需要进行的分配 (Distribute) 和收集 (Collect) 的总趟数分别是 ()
- A. 1 趟分配, 1 趟收集
 - B. 3 趟分配, 3 趟收集
 - C. 10 趟分配, 10 趟收集
 - D. 3 趟分配, 10 趟收集
9. 在对海量文件进行外部排序 (External Sorting) 的优化过程中, 理论上增加归并路数 m 可以有效减少归并的总趟数, 从而大幅削减极其耗时的磁盘 I/O 次数。然而, 在实际工程实现中, 归并路数 m 绝对不是越大越好, 盲目增大 m 带来的最大副作用是 ()
- A. 会导致第一阶段生成的初始归并段的长度被迫大幅度缩短
 - B. 会导致内部归并时在 m 个归并段中选择最小关键字的比较次数激增, 使得 CPU 运算时间反演为系统瓶颈
 - C. 会使得最佳归并树的高度不降反升, 直接引发磁盘 I/O 次数不合常理地增加
 - D. 外部排序的空间复杂度会随着 m 的增大而呈现指数级的恶性上升
10. 在数据结构所有主流内部排序算法中, 某种算法在最坏情况下的时间复杂度依然能够稳定在 $O(n \log_2 n)$, 同时它是一种不稳定的排序算法, 并且其辅助空间复杂度为 $O(1)$ 。具备上述所有核心物理特征的算法是 ()
- A. 快速排序 (Quick Sort)
 - B. 希尔排序 (Shell Sort)
 - C. 堆排序 (Heap Sort)
 - D. 归并排序 (Merge Sort)
11. 在为华为昇腾 (Ascend) NPU 开发底层的向量化排序算子时, 工程师采用快速排序 (Quick Sort) 对海量的特征张量进行原地排序。为了极致压榨硬件性能, 该算子在 NPU 核心极其有限的局部内存 (Local Memory) 中分配了固定大小的函数调用栈。在运行某一批“特征极其稀疏且存在大量连续重复零值”的张量数据时, NPU 核心突然发生 `stack overflow` (栈溢出) 崩溃。导致该崩溃的最本质原因是 ()
- A. 快速排序是不稳定的排序算法, 大量重复的零值导致元素位置频繁交换, 引发内存越界
 - B. 当数组元素基本有序或存在大量重复元素时, 若未优化枢轴 (Pivot) 选择, 快速排序的递归树将严重向一侧倾斜, 导致空间复杂度退化为 $O(n)$, 撑爆有限的硬件栈
 - C. 快速排序在处理大量零值时, 其时间复杂度会退化为 $O(n^2)$, 导致 NPU 看门狗超时复位
 - D. 在原生的快速排序中, Partition 划分过程需要额外申请 $O(n)$ 的辅助数组来暂存数据, 导致局部内存耗尽

12. 在大语言模型（如 DeepSeek-V3）的混合专家（MoE）架构中，路由网络需要为当前输入的 Token 从 256 个备选专家网络中，快速挑选出打分最高（概率最大）的前 4 个专家（即 Top-4 问题）
- 为了将算法的时间开销降到最低，系统底层的 C++ 引擎决定采用堆（Heap）这一数据结构来优化查找过程。关于该场景下的堆操作，以下在理论上最优的设计是（ ）
- A. 将 256 个专家的打分构建成一个包含 256 个元素的最大堆（Max-Heap），建堆的时间复杂度为 $O(n \log n)$ ，随后执行 4 次输出堆顶并调整的操作
 - B. 维护一个容量仅为 4 的最小堆（Min-Heap）。遍历后续的 252 个专家打分，若大于堆顶元素，则替换堆顶并进行“向下调整”。最终堆中保留的就是 Top-4
 - C. 将 256 个专家的打分构建成一个包含 256 个元素的最小堆（Min-Heap），然后直接取堆底的 4 个叶子节点即可
 - D. 采用二路归并排序对 256 个专家进行完全排序，因为归并排序的比较次数最少，寻找 Top-4 的整体时间复杂度最低
13. 在分析数亿条出行记录时，需要将散落在几十个按月存储的 CSV 文件合并，并按（取车时间，经纬度）进行全局排序。由于数据量高达数百 GB，无法全部装入内存，分析师采用了多路归并的外部排序（External Sorting）
- 为了减少昂贵的磁盘读写次数，分析师决定将归并的路数 k 从 2 路大幅提升到 16 路。以下推论绝对正确的是（ ）
- A. 增加归并路数 k 可以减少初始归并段的个数，从而直接降低总的磁盘 I/O 次数
 - B. 归并趟数 $S = \lceil \log_k r \rceil$ （ r 为初始段个数）。增大 k 会显著减少归并趟数，从而减少磁盘 I/O，但如果引入“败者树（Loser Tree）”，内存中比较关键字的 CPU 耗时会大幅增加
 - C. 无论 k 增加到多大，由于每次归并都需要把数据读入内存再写回磁盘，总的磁盘 I/O 次数只与原始数据总量相关，与 k 无关
 - D. 在 16 路归并中，为了选出最小的记录，使用败者树进行比较的次数会多于传统的直接比较方法，但胜在结构稳定
14. 在分布式共识算法（如 Raft）中，某个从节点（Follower）由于短暂的网络抖动，刚刚接收到主节点补发的一小批日志条目。此时，该从节点内存中的千万级日志数组绝大部分已经是严格按时间戳递增排序的，只有尾部新追加的几十条日志存在轻微的乱序
- 如果此时需要调用一个算法将整个日志数组彻底排序，在不考虑额外空间开销的前提下，以下排序算法中执行效率最高的是（ ）
- A. 直接插入排序（Straight Insertion Sort）
 - B. 简单选择排序（Simple Selection Sort）
 - C. 快速排序（Quick Sort）
 - D. 二路归并排序（Merge Sort）
15. 币安（Binance）的后台安全系统需要对 2026 年上半年的海量数字货币交易流水进行排序审查。每条流水包含三个独立的整数维度：[年份 Year]（只有 2026）、[月份 Month]（1-6）、[交易额 Amount]（0-99999999 的整数）
- 现要求审查报表必须先按 Year 升序，如果 Year 相同则按 Month 升序，如果 Month 相同再按 Amount 升序
- 工程师决定采用最低位优先（LSD）的基数排序（Radix Sort）思想，通过调用底层的“桶排序（Bucket Sort）”模块进行多次分配与收集。关于该算法的设计，以下说法正确的是（ ）
- A. LSD 的执行顺序必须是：第 1 趟按 Year 桶排 → 第 2 趟按 Month 桶排 → 第 3 趟按 Amount 桶排
 - B. 作为子过程的底层桶排序模块，必须使用稳定（Stable）的排序算法或分配收集机制，否则后续趟次的排序会破坏前一趟排好的内部顺序
 - C. 由于 Amount 的取值范围极大（0-99999999），基数排序在这种场景下的时间复杂度必定退化为 $O(n^2)$ ，不如直接使用快速排序
 - D. LSD 的执行顺序必须是：第 1 趟按 Amount 桶排 → 第 2 趟按 Month 桶排 → 第 3 趟按 Year 桶排，且子模块可以是不稳定排序

16. 【2024 统考真题】使用快速排序算法对含 n ($n \geq 3$) 个元素的数组 M 进行排序, 若第一趟排序将 M 中除枢轴外的 $n-1$ 个元素划分为均不为空的 P 和 Q 两块, 则下列叙述中, 正确的是 ()。
- A. P 与 Q 块间有序 B. P 与 Q 均块内有序
C. P 和 Q 的元素个数大致相等 D. P 中和 Q 中均不存在相等的元素
17. 【2024 统考真题】已知关键字序列 28, 22, 20, 19, 8, 12, 15, 5 是大根堆 (最大堆), 对该堆进行两次删除操作后, 得到的新堆为 ()。
- A. 20, 19, 15, 12, 8, 5 B. 20, 19, 15, 5, 8, 12
C. 20, 19, 12, 15, 8, 5 D. 20, 19, 8, 12, 15, 5
18. 【2024 统考真题】现有由关键字组成的 3 个有序序列 (3, 5)、(7, 9) 和 (6), 若按从左至右的次序选择有序序列进行 2 路归并排序, 则关键字之间的总比较次数是 ()。
- A. 3 B. 4 C. 5 D. 6
19. 【2025 统考真题】在下列排序算法中, 最坏情况下元素移动次数最少的是 ()。
- A. 起泡排序 B. 直接插入排序 C. 快速排序 D. 简单选择排序
20. 【2025 统考真题】对含 9 个关键字的初始序列进行排序, 若序列的变化情况如下表所示, 则下列排序算法中, 采用的是 ()。

初始序列	5, 25, 40, 30, 10, 20, 45, 15, 35
第 1 趟排序后的序列	5, 10, 20, 30, 15, 35, 45, 25, 40
第 2 趟排序后的序列	5, 10, 15, 25, 20, 30, 40, 35, 45

- A. 希尔排序 B. 基数排序 C. 归并排序 D. 折半插入排序
21. 【2024 统考真题】在外排序中, 利用败者树对初始为升序的归并段进行多路归并, 败者树中记录“冠军”的结点保存的是 ()。
- A. 最大关键字 B. 最小关键字
C. 最大关键字所在的归并段号 D. 最小关键字所在的归并段号

22.

2021年真题

应用题

42. (8 分) 已知某排序算法如下:

```
void cmpCountSort(int a[], int b[], int n)
{
    int i, j, *count;
    count = (int *)malloc(sizeof(int)*n);    //C++语言: count=new int[n];
    for(i=0; i<n; i++)    count[i]=0;
    for(i=0; i<n-1; i++)
        for(j=i+1; j<n; j++)
            if(a[i]<a[j])    count[j]++;
            else    count[i]++;
    for(i=0; i<n; i++)    b[count[i]] = a[i];
    free(count);    //C++语言: delete count;
}
```

请回答下列问题。

- 若有 `int a[] = {25, -10, 25, 10, 11, 19}, b[6];`, 则调用 `cmpCountSort(a, b, 6)` 后数组 `b` 中的内容是什么?
- 若 `a` 中含有 n 个元素, 则算法执行过程中, 元素之间的比较次数是多少?
- 该算法是稳定的吗? 若是, 则阐述理由; 否则, 修改为稳定排序算法。

23.

42. (10分) 现有 n ($n > 100\,000$) 个数保存在一维数组 M 中, 需要查找 M 中最小的 10 个数。请回答下列问题。

- (1) 设计一个完成上述查找任务的算法, 要求平均情况下的比较次数尽可能少, 简述其算法思想 (不需要程序实现)。
- (2) 说明你所设计的算法平均情况下的时间复杂度和空间复杂度。

24.

2023年真题

42. (10分) 对含有 n ($n > 0$) 个记录的文件进行外部排序, 采用置换-选择排序生成初始归并段时需要使用一个工作区, 工作区中能保存 m 个记录。请回答下列问题。

- 1) 若文件中含有 19 个记录, 其关键字依次是 51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100, 则当 $m = 4$ 时, 可生成几个初始归并段? 各是什么?
- 2) 对任意的 m ($n \gg m > 0$), 生成的第一个初始归并段的长度最大值和最小值分别是多少?

25.

04. 【2016 统考真题】已知由 n ($n \geq 2$) 个正整数构成的集合 $A = \{a_k | 0 \leq k < n\}$, 将其划分为两个不相交的子集 A_1 和 A_2 , 元素个数分别是 n_1 和 n_2 , A_1 和 A_2 中的元素之和分别为 S_1 和 S_2 . 设计一个尽可能高效的划分算法, 满足 $|n_1 - n_2|$ 最小且 $|S_1 - S_2|$ 最大。要求:

- 1) 给出算法的基本设计思想。
- 2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- 3) 说明所设计算法的平均时间复杂度和空间复杂度。

26. 关于各内部排序算法在特定数据状态下的表现, 下列叙述中正确的有 () 个

- I. 对 N 个记录进行快速排序, 在最坏情况下其时间复杂度退化为 $O(N^2)$, 此时其空间复杂度依然可以保持为 $O(\log_2 N)$
- II. 堆排序在最好、最坏和平均情况下的时间复杂度均为 $O(N \log_2 N)$, 且其在建堆 (Build-Heap) 阶段所需的时间复杂度为 $O(N)$
- III. 若初始序列完全逆序, 则起泡排序 (bubble sort) 和直接插入排序所需的比较次数和移动次数均达到最大值
- IV. 希尔排序 (shell sort) 的最后一趟实际上就是增量为 1 的直接插入排序, 因此希尔排序也是一种稳定的排序算法

- A. 1
- B. 2
- C. 3
- D. 4

27. 以下关于比较类排序算法运行过程的说法中, 正确的是 ()

- A. 二路归并排序 (merge sort) 在合并两个长度分别为 m 和 n 的有序表时, 最坏情况下的比较次数为 $m + n$, 最好情况下的比较次数为 $\min(m, n)$
- B. 在简单选择排序中, 第一趟排序结束后, 最小 (或最大) 的元素一定已经到达其最终位置, 且任何情况下该算法的总比较次数均为 $N(N-1)/2$
- C. 判断一个序列是否为某趟排序的中间结果, 若发现序列的前半部分已经完全有序, 且后半部分元素均大于前半部分, 则该状态必定是由快速排序产生的
- D. 若使用基于比较的算法对 N 个元素进行排序, 只要算法设计得足够优秀, 其最坏情况下的时间复杂度下界可以突破 $O(N \log_2 N)$, 达到 $O(N)$

28. 在进行外部排序时, 为了优化磁盘 I/O 并提高整体排序效率, 通常会引入多路归并、败者树 (Loser Tree) 以及置换-选择排序算法。关于这些技术的协同作用, 下列推论中正确的是 ()

- I. 引入败者树的主要目的是为了减少多路归并时的磁盘读写次数
- II. 采用置换-选择排序算法生成的初始归并段长度是可变的，其平均长度通常约为内存工作区大小 (W) 的 2 倍，这能够有效减少初始归并段的数量
- III. 增加归并路数 k 可以减少归并趟数 S，因此在可用内存总量固定的情况下，盲目增大 k 值必定能使得整个外部排序的总时间大幅缩短
- IV. 在进行 k 路严格树形归并时，若初始归并段数量为 M，且 $(M - 1) \pmod{k - 1} \neq 0$ ，则必须添加长度为 0 的“虚段”才能构造出严格的 k 叉树

- A. 仅 I、II
- B. 仅 II、IV
- C. 仅 II、III、IV
- D. I、II、IV

29. 已知一个长度为 n 的整数数组 $A[0 \dots n-1]$ ，其中的元素仅包含三种取值：0、1、2。请设计一个尽可能高效的原地 (In-place) 排序算法，将数组重新排列，使得所有的 0 出现在数组最前面，所有的 1 出现在中间，所有的 2 出现在数组最后

请完成下列任务：

- 1. 给出算法的基本设计思想
- 2. 采用 C 或 C++ 语言描述算法，关键之处给出注释
- 3. 分析你所设计算法的时间复杂度和空间复杂度

30. 直接插入排序

折半插入排序

希尔排序

冒泡排序

快速排序

简单选择排序

堆排序

归并排序

计数排序(看一下实现即可，不要求手写)